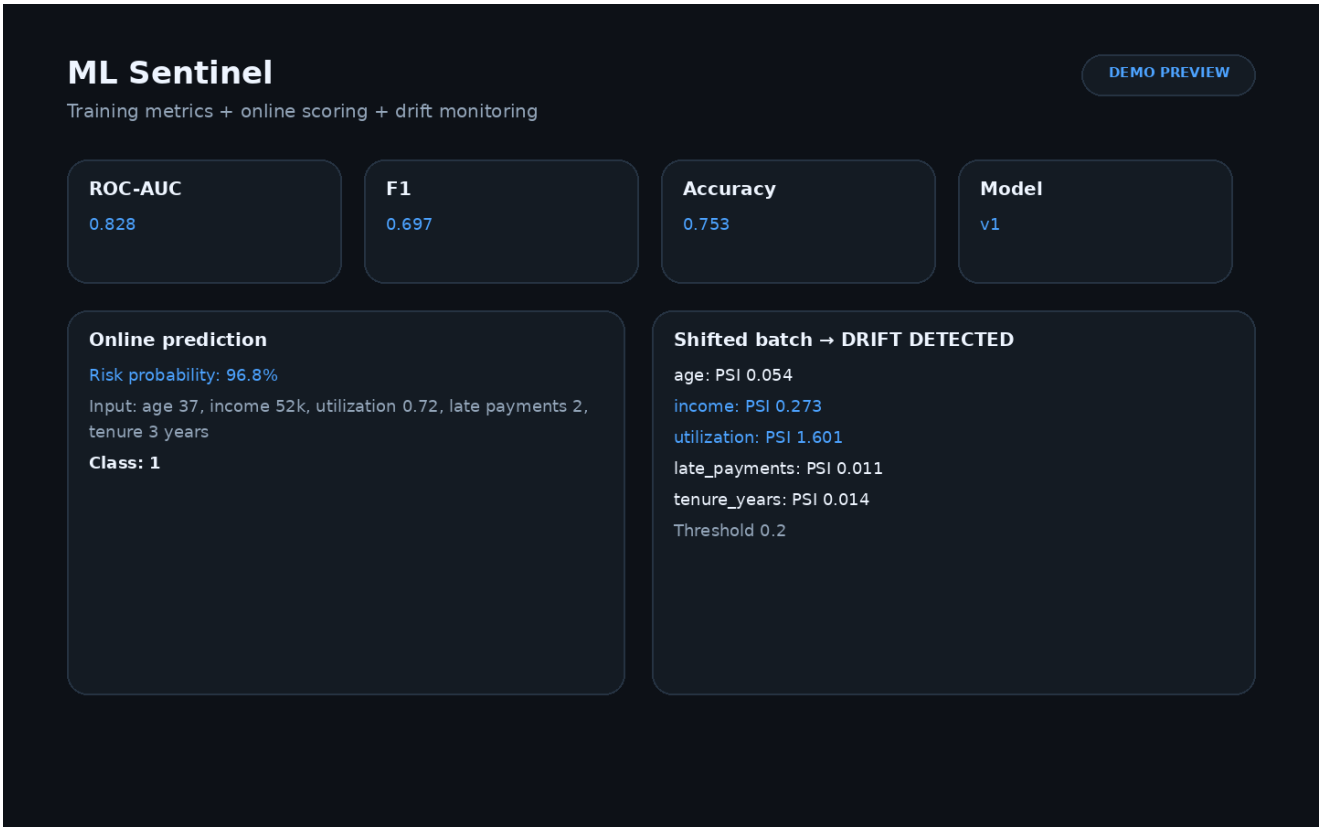
ML Sentinel

MLOps & Model Monitoring Platform

Python • Scikit-learn • FastAPI • PSI Drift Monitoring • Docker • GitHub Actions • MLflow/Airflow-ready



Representative demo preview. Run the included application for the interactive version.

1. Problem and Solution

A trained model can degrade after deployment when the incoming population changes. ML Sentinel demonstrates the model lifecycle: training, versioned quality metrics, online inference, data-drift checks and retraining.

Why this is strong on an AI/ML resume

The project is built as a small product rather than only a notebook. It includes a backend API, interactive frontend, repeatable local data, automated tests, Docker packaging, CI configuration, demo instructions, and clear model/engineering boundaries.

Core components

Component	Responsibility
Training Pipeline	Scales features and trains a reproducible classification model.
Model Metrics	Tracks accuracy, F1 and ROC-AUC with a model version.
Prediction Service	Validates online records and returns class probabilities.
Drift Monitor	Compares incoming feature distributions to training baselines using PSI and raises an alert beyond a threshold.

2. Architecture and Data Flow

Reference distributions + Incoming batch → PSI Monitor → Drift Status → Alert / Retraining decision

Optional: MLflow tracking + Airflow retraining schedule + container deployment + CI.

Engineering decisions

- Keep the core demo fully local so it is reliable during interviews.
- Separate model logic from API/UI so algorithms can be upgraded without rewriting the product surface.
- Use explicit validation and automated tests so the demo can be explained as an engineering system, not only an ML experiment.
- Package with Docker and CI configuration to demonstrate repeatability.

What “production-style” means here

The repositories demonstrate production patterns, but they are portfolio systems, not claims of enterprise production deployment. A real deployment would add identity/access management, secret management, persistent observability, load testing, threat modeling, data governance, backup/rollback processes and infrastructure-specific scaling.

3. Live Demo Walkthrough

1. Show the model quality cards and current version.

2. Run an online prediction and explain probability vs threshold.

3. Check a stable batch; PSI remains below 0.2.

4. Check a shifted batch; drift alert appears.

5. Retrain and show the model version increment; explain MLflow/Airflow production integration.

What to say while demoing

> *ML Sentinel focuses on the part of machine learning that starts after model training. The service trains a classification pipeline, records quality metrics and a model version, exposes online inference, and compares incoming feature distributions against the training baseline using PSI. I can simulate a stable batch and a shifted batch; once PSI crosses 0.2 the dashboard raises a drift alert. I also included retraining, Docker/CI and optional MLflow/Airflow adapters.*

Demo failure fallback

Keep `demo_output.json` and the preview image in the repository. If a live environment fails, walk through the saved output, explain the expected API call, then show the unit tests. This demonstrates preparedness without pretending the failed live run succeeded.

4. Resume Bullets

- Built an end-to-end MLOps service covering data preprocessing, model training, versioned metrics, online inference, retraining and production-style drift monitoring using Population Stability Index (PSI).
- Exposed model-health and prediction APIs with FastAPI, input validation, automated tests, Docker packaging and GitHub Actions CI for repeatable deployment.
- Added optional MLflow experiment-tracking and Airflow retraining integrations, with a modular design for model registry, scheduled retraining and alerting workflows.

Important claim rule

Only claim technologies you can run and explain. Optional modules such as YOLO, PyTorch two-tower models, FAISS, MLflow or Airflow are included as upgrade paths where noted; do not say you trained/deployed them until you actually execute that path and record evidence/metrics.

Metrics to discuss

ROC-AUC / F1	Prediction latency	PSI by feature	Performance after drift	Retraining/rollback success

5. Interview Questions

Why PSI?

It is simple, interpretable and commonly used for monitoring tabular feature distribution shifts. It should be combined with model-quality metrics when labels arrive.

When should retraining happen?

Not on drift alone. Use drift severity, business impact, enough new labeled data, performance degradation and approval/rollback policies.

Questions you should be ready for

- Why did you choose this algorithm instead of an alternative?
- What is the biggest failure mode in the current implementation?
- How would the architecture change at 100x traffic/data size?
- Which offline and online metrics would you monitor?
- What did you personally implement, test and debug?

6. Upgrade Roadmap

- Add authentication and role-based access.
- Persist model/retrieval artifacts with explicit versioning.
- Add structured observability: request logs, latency, error rates and model metrics.
- Create a small evaluation suite with expected outputs and regression thresholds.
- Deploy to a cloud service and record the public demo URL in the README.
- Add one real dataset experiment and document the training/evaluation results.

Final presentation advice

Do not memorize only the feature list. Be able to draw the data flow, explain one algorithm choice, show one failure case, quote one meaningful metric and describe one scalability improvement. That is what turns the project from a resume line into an interview asset.